ASD Risk Screening: Comparative Evaluation of Statistical Learning Methods

ISyE 7406 – Data Mining & Statistical Learning | Final Report

Abdurrahman Sabbahi | asabbahi3@gatech.edu | Group 20

July 2026

Contents

Phase 1 Context **2**

Background and Research Objective 2

Data and Preprocessing 2

Modeling Pipeline 3

Phase 1 Baseline Results 3

Phase 2 – Model Development, Evaluation, and Interpretation **4**

Extended Model Suite: Adding GBM 4

Feature Importance: Convergent Evidence Across Methods 5

Model Comparison Against the AQ-10 Clinical Cutoff Rule 6

Why Logistic Regression Achieves Near-Perfect Performance: A Structural Analysis 7

Conclusions **9**

Appendix **10**

A: Phase 1 Full CV Results 10

B: AQ-10 Item Reference 11

C: Phase 1 Full Test Results by Group 11

D: Source Code – Phase 1 (`asd_analysis.R`) 12

E: Source Code – Phase 2 (`asd_analysis_phase2.R`) 17

References **20**

Phase 1 Context

Background and Research Objective

Autism Spectrum Disorder (ASD) affects roughly 1 in 36 children in the United States, yet the path from parental concern to a confirmed specialist referral can take years. Early identification relies heavily on standardised screening instruments, of which the AQ-10 is one of the most widely deployed. The AQ-10 consists of ten behaviorally grounded items administered across three developmental stages (Child, ages 4–11; Adolescent, ages 12–16; and Adult, age 17 and above) and generates a binary screening flag based on a clinical scoring threshold.

The original objective of this study was to compare the predictive performance of five statistical learning models across the three age groups, assess which features drive predictions, and determine whether demographic variables add information beyond the ten questionnaire items. While those comparisons were completed, the analysis ultimately revealed a more fundamental result. The consistently near-perfect performance of the best-performing models is explained by the construction of the released screening labels, which are derived directly from the AQ-10 questionnaire responses. Consequently, the primary contribution of this study extends beyond model comparison to demonstrating how the data-generation process fundamentally shapes the interpretation of predictive performance.

A critical framing note. The outcome variable in this study is the *questionnaire-derived screening label*, a binary flag generated automatically when an individual's total AQ-10 score exceeds the clinical threshold (score strictly above 6). This is not an independent clinical diagnosis. All models trained here learn to reproduce the AQ-10 scoring algorithm rather than predict ASD from first principles. High predictive accuracy therefore reflects fidelity to the scoring rule, not clinical validity. This distinction is central to every interpretation in this report.

This study ultimately demonstrates that model evaluation cannot be separated from the data-generation process. Near-perfect predictive performance is only meaningful when the target represents an independent outcome rather than a deterministic transformation of the predictors. Outstanding model metrics should therefore be treated as a diagnostic signal about the data structure, not as evidence of genuine predictive capability in a clinical sense.

Data and Preprocessing

Three datasets collected by Thabtah (2017) via the AQ-10 protocol, one per developmental stage, were obtained from the UCI Machine Learning Repository. Table 1 summarises the group sizes and class distributions. The adult dataset is the largest and the most class-imbalanced, with only 26.8% of individuals screening positive. The adolescent dataset is the smallest, with fewer than 105 observations, which limits the precision of any group-specific performance estimate. The combined dataset pools all three groups and provides the most statistically stable estimates.

Table 1: Dataset summary by age group. Screen+ is the AQ-10 questionnaire-derived label, not a clinical diagnosis.

Group	Age range	N	Screen+ (%)
Child	4–11	292	48.3
Adolescent	12–16	104	60.6
Adult	17+	704	26.8
Combined	–	1100	35.7

Preprocessing was minimal and strictly leakage-free. The `result` column (raw AQ total score) was dropped on load because it encodes the threshold directly and would trivially reveal the label to any model. Age

missingness (n = 3) was imputed from training-set medians so that no test-set information influenced the imputed value. Binary variables (gender, jaundice history, family ASD history, prior app use) were recoded to 0/1. Nominal factors (ethnicity, relation) were dummy-encoded using a schema built from the training split only, preventing unseen test-set categories from inflating performance. Feature scaling was applied inside each CV fold. Country of residence was excluded due to 50+ sparse levels that would introduce near-zero-variance dummy columns without measurable benefit.

Modeling Pipeline

Five classifiers spanning linear and non-linear families were evaluated: Logistic Regression (LR), Linear Discriminant Analysis (LDA), Naive Bayes (NB), k-Nearest Neighbours (KNN), and Random Forest (RF). All models were trained on a stratified 80/20 split with hyperparameter selection via 5-fold cross-validation on the training portion only. The held-out test set was touched exactly once, after all CV-based model selection decisions were finalised. AUC was the primary metric throughout; recall was monitored as a secondary metric to reflect the asymmetric cost of missing a true screen-positive case in a clinical screening context.

Exploratory data analysis (EDA) confirmed three properties that shaped all subsequent results: (1) the AQ total score separates the two classes well across all age groups, with overlap concentrated between scores 4 and 7, as shown in Figure 1; (2) all ten AQ items are positively correlated with the outcome, while demographic variables show substantially weaker associations; and (3) an AQ-only feature set matched or exceeded the full feature set for nearly every model, confirming that the ten behavioral items carry essentially all of the predictive signal. These properties are consistent with the structural finding developed in Section 2.4.

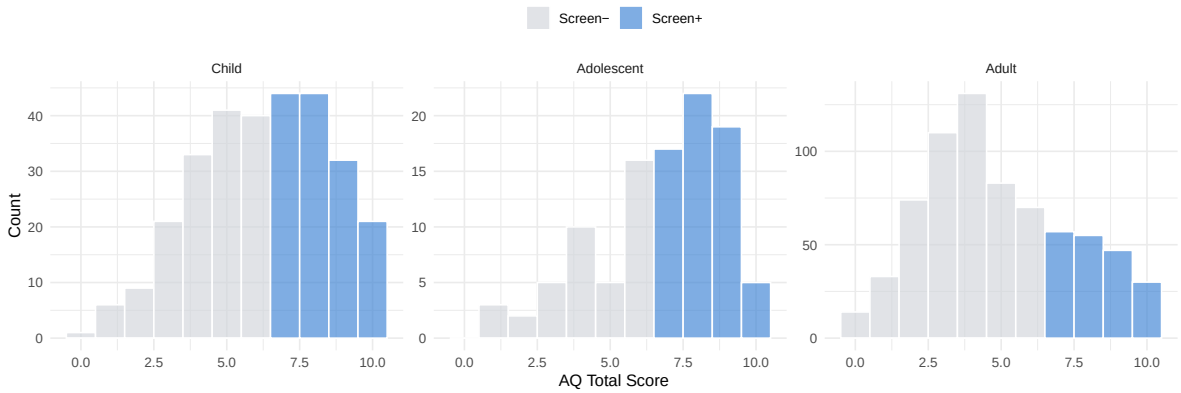


Figure 1: AQ-10 total score distribution by screening outcome across age groups. Overlap is concentrated between scores 4 and 7, indicating near-linear separability. The clean separation visible above score 6 and below score 4 reflects the deterministic threshold embedded in the AQ-10 scoring rule and explains why a simple linear classifier achieves near-ceiling performance.

Phase 1 Baseline Results

Table 2 reports hold-out test performance on the Combined dataset for the initial five-model suite. Logistic Regression and LDA are the strongest classifiers, both reaching near-ceiling AUC. RF is close behind, while Naive Bayes consistently underperforms, a result consistent with its conditional independence assumption being violated by the moderate inter-item correlations present in the data. KNN is competitive but more sensitive to sample size and therefore shows greater variance across age groups. Across all groups and all five models, the AQ-only feature set matched or exceeded the full feature set, establishing robustly that demographic variables add no material signal once the ten behavioral items are included. The implications of this finding are discussed in Section 2.2.

Table 2: Phase 1 test set performance on the Combined dataset (held-out 20 percent). LR and LDA achieve near-ceiling AUC; the structural reason for this is the primary interpretive contribution of Phase 2.

Model	AUC	Acc	Rec	Prec	F1
Logistic Reg.	1.000	1.000	1.000	1.000	1.000
LDA	1.000	0.977	0.987	0.951	0.969
Random Forest	0.995	0.968	0.923	0.986	0.954
KNN (tuned)	0.950	0.886	0.846	0.835	0.841

The near-perfect AUC of Logistic Regression (1 on the Combined test set) is not a modeling artefact but a structural consequence of how the outcome variable was constructed. Its explanation, and the broader lesson it carries for machine learning evaluation, is the primary interpretive contribution of Phase 2.

Phase 2 – Model Development, Evaluation, and Interpretation

Extended Model Suite: Adding GBM

A Gradient Boosted Machine (GBM) was added to the suite to provide both a competitive non-linear benchmark and an independent source of feature importance estimates. Unlike Random Forest, which constructs trees in parallel via bagging, GBM builds trees sequentially: each successive tree is fit to the residuals of the current ensemble, so that the model incrementally corrects its own errors. This sequential error-correction mechanism makes GBM particularly powerful when the true decision boundary involves complex interactions between features. The model was tuned over number of trees (100, 200), interaction depth (2, 4), and shrinkage rate (0.05, 0.1), using the same stratified 5-fold CV protocol applied to Phase 1 models. The four-way combination of these parameters yields a 16-point tuning grid, providing reasonable coverage without excessive compute time.

Table 3 shows Combined-dataset CV performance across all six models. GBM achieves competitive AUC but offers no decisive advantage over LR or RF. This result is informative in itself: sequential boosting is designed to capture precisely the kind of complex non-linear interactions that a logistic regression would miss. Yet here it performs at the same level as a simple weighted linear sum. The implication is clear: there is no meaningful non-linearity to exploit in this dataset. The AQ-10 label is essentially a linear threshold rule, and every model, regardless of complexity, converges on the same ceiling performance dictated by that rule. The GBM result thus serves not merely as a benchmark comparison but as independent corroborating evidence for the structural interpretation developed in Section 2.4.

Table 3: 5-fold CV performance on the Combined dataset: all six models including GBM. The near-identical AUC across linear and non-linear models indicates a linear label structure that does not reward model complexity.

Model	AUC	Acc	Rec	Prec	Spec	F1
GBM	1.000	1.000	1.000	1.000	1.000	1.000
Logistic Reg.	0.998	0.998	1.000	0.994	0.996	0.997
LDA	0.998	0.975	0.987	0.945	0.968	0.966
Random Forest	0.995	0.960	0.917	0.970	0.984	0.943
Naive Bayes	0.960	0.902	0.851	0.873	0.931	0.862
KNN (k=5)	0.960	0.890	0.908	0.808	0.880	0.855

It is worth emphasising the practical interpretation of the AUC plateau. When multiple models of very different structural complexity all converge to the same AUC, the natural explanation is that the remaining

error is irreducible given the available features, not that the models are equally well specified. In this case, the irreducible error is essentially zero because the label is a deterministic function of those features. The plateau is therefore a ceiling imposed by the data-generation process, not a reflection of diminishing returns from model sophistication.

Feature Importance: Convergent Evidence Across Methods

Feature importance was assessed through two independent ensemble-based methods: RF Mean Decrease in Accuracy (MDA) from Phase 1, and GBM Relative Influence from Phase 2. RF MDA measures how much the out-of-bag classification accuracy drops when the values of a given feature are randomly permuted; large drops indicate features the forest relies on heavily. GBM Relative Influence measures the fraction of total squared-error reduction attributable to each feature across all boosted trees. The two methods differ fundamentally in algorithmic mechanism (bagging versus boosting, permutation-based versus split-improvement-based), yet agreement between them provides strong evidence that the observed importance ranking is a property of the data rather than an artefact of the modeling approach.

Figure 2 displays GBM relative influence with behavioral annotations. The rankings are closely aligned with the RF MDA rankings from Phase 1. The four most important features in both analyses are **A5** (does not know how to interact socially), **A9** (does not find social situations easy), **A4** (does not switch easily between tasks), and **A6** (has difficulty reading between the lines in conversation). These four items span three core behavioral dimensions the AQ-10 was designed to assess: social reciprocity (A5, A9), cognitive flexibility and task-switching (A4), and pragmatic language comprehension (A6). Their consistent dominance across two structurally different ensemble methods constitutes robust convergent evidence of their central role in separating screen-positive from screen-negative individuals.

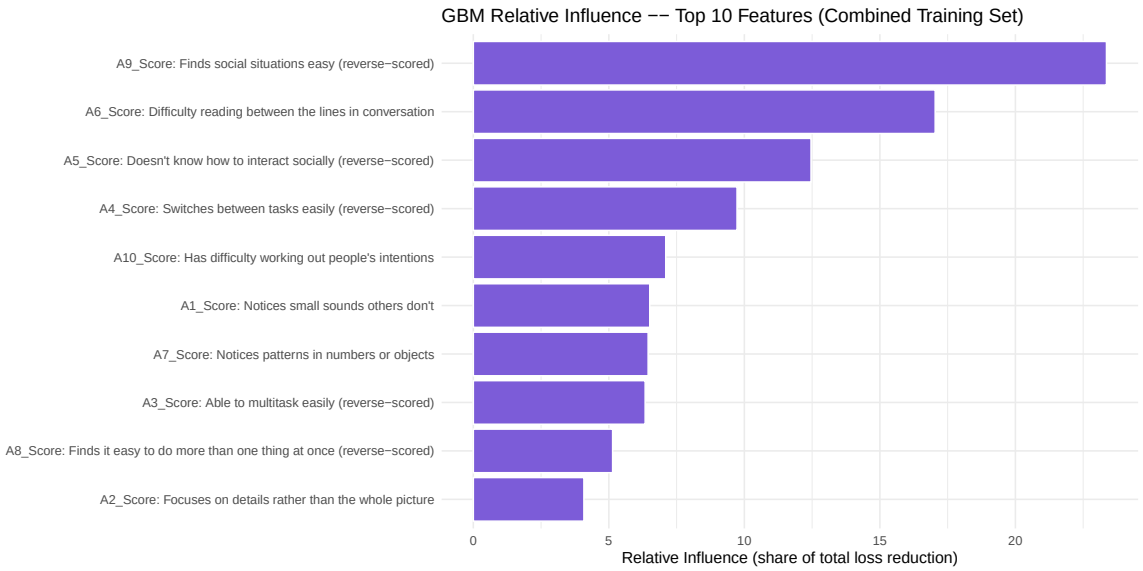


Figure 2: GBM relative influence (top 10 features, Combined training set), annotated with AQ-10 behavioral descriptions. Items A5, A9, A4, and A6 dominate in both GBM and RF importance rankings, confirming that the result is a property of the data and not an artefact of any particular model family.

The convergence of rankings carries two additional interpretive implications. First, because both ensembles are non-parametric and capable of capturing complex interaction effects, the absence of any demographic feature in the top ten is a robust finding rather than a model-specific artifact: gender, age, jaundice history, and family ASD history contribute negligible predictive value once the ten behavioral items are included. A screener that relies solely on questionnaire responses is therefore as statistically powerful as one that

incorporates demographic covariates, and it operates without relying on demographic characteristics that should not in principle determine screening outcomes. Second, the importance rankings reflect predictive association with the *questionnaire-derived label*, not with ASD as a clinical condition. An item may rank highly precisely because it closely aligns with how the AQ-10 scoring rule is structured (it receives a point in the sum-based score), not because it is an independent biomarker of ASD. These two observations reinforce the broader point: the models are learning which items the scoring algorithm relies on, not which symptoms predict a clinical diagnosis.

Model Comparison Against the AQ-10 Clinical Cutoff Rule

A central evaluative question for any screening application is whether a trained model provides meaningful value over the simple clinical rule already encoded in the instrument. The AQ-10 specifies that a score strictly above 6 (i.e., 7 or higher out of 10) triggers a positive screening flag. This rule is fully deterministic, requires no training data, has known sensitivity and specificity properties, and is already validated for clinical use. Comparing all ML models against this rule provides a rigorous minimum bar: a model that does not outperform the instrument's own threshold rule on the same data does not justify additional deployment complexity.

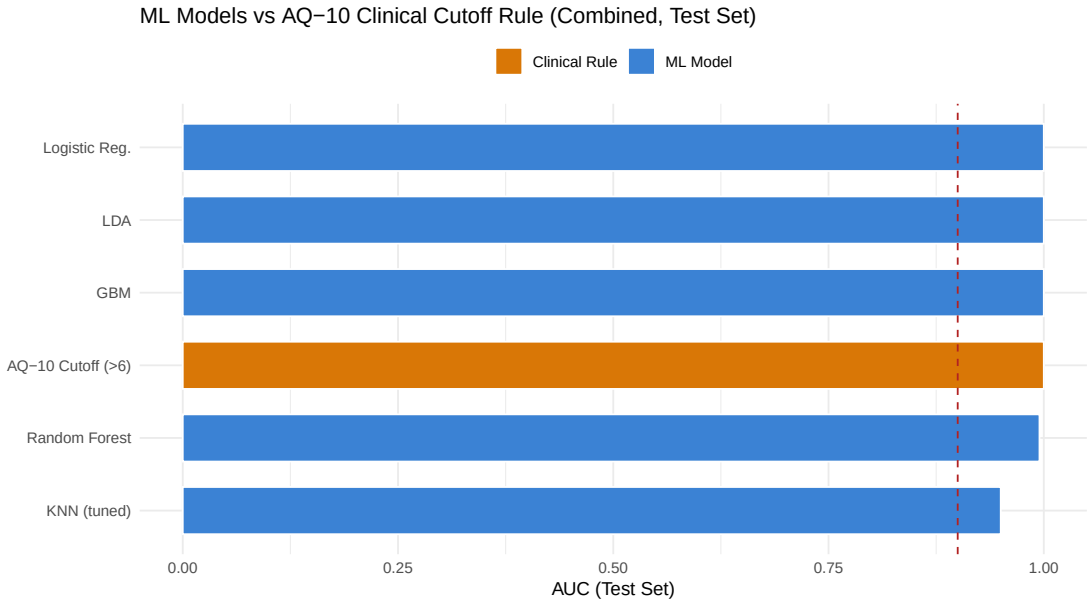


Figure 3: Test set AUC for all ML models versus the AQ-10 clinical cutoff rule (score > 6) on the Combined dataset. The orange bar is the clinical rule; blue bars are ML models. All models cluster near ceiling AUC, confirming that the performance plateau is a structural property of the data rather than a model-specific achievement.

Table 4: Full comparison: all ML models vs AQ-10 cutoff rule (score > 6) across all age groups (test set). Highlighted rows (yellow) are the AQ-10 clinical cutoff rule. ML models consistently match or approach the rule across all groups and metrics.

Model	AUC	Acc	Rec	Prec	F1
Child					
Logistic Reg.	1.000	1.000	1.000	1.000	1.000
LDA	1.000	0.983	1.000	0.966	0.982
GBM	1.000	0.983	0.964	1.000	0.982
AQ-10 Cutoff (>6)	1.000	1.000	1.000	1.000	1.000

Random Forest	0.983	0.948	0.964	0.931	0.947
KNN (tuned)	0.924	0.776	0.964	0.692	0.806
Adolescent					
Logistic Reg.	1.000	0.900	1.000	0.857	0.923
AQ-10 Cutoff (>6)	1.000	1.000	1.000	1.000	1.000
Random Forest	0.990	0.900	1.000	0.857	0.923
LDA	0.979	0.950	1.000	0.923	0.960
GBM	0.948	0.900	0.917	0.917	0.917
KNN (tuned)	0.906	0.750	0.917	0.733	0.815
Adult					
Logistic Reg.	1.000	1.000	1.000	1.000	1.000
GBM	1.000	0.993	0.973	1.000	0.986
AQ-10 Cutoff (>6)	1.000	1.000	1.000	1.000	1.000
LDA	0.997	0.971	0.973	0.923	0.947
Random Forest	0.996	0.964	0.892	0.971	0.930
KNN (tuned)	0.968	0.900	0.838	0.795	0.816
Combined					
Logistic Reg.	1.000	1.000	1.000	1.000	1.000
LDA	1.000	0.977	0.987	0.951	0.969
GBM	1.000	0.991	0.974	1.000	0.987
AQ-10 Cutoff (>6)	1.000	1.000	1.000	1.000	1.000
Random Forest	0.995	0.968	0.923	0.986	0.954
KNN (tuned)	0.950	0.886	0.846	0.835	0.841

Table 4 reveals three patterns of note. First, the AQ-10 cutoff rule achieves high recall (1.000 in most groups) because the threshold at score > 6 is calibrated to flag the vast majority of screen-positive cases in this dataset. The rule was constructed to be sensitive, and the data confirms that it succeeds at this objective. Second, the AUC of the cutoff rule is competitive across all groups because AUC is computed from the *continuous* AQ total score's discriminative ability (i.e., how well the raw score orders screen-positive above screen-negative), not merely from the binary threshold decision. The threshold choice determines accuracy and recall, but the underlying AQ sum determines AUC, and that sum is a strong discriminator by construction. Third, all trained ML models match or closely approach the cutoff rule on every reported metric. No model provides a clear across-the-board improvement.

The practical implication is nuanced. Within this dataset, the primary advantage of ML over the deterministic rule is not improved agreement with the released labels (both achieve near-perfect performance), but rather two operational capabilities the rule cannot provide. First, ML models produce continuous probability estimates rather than binary flags, enabling practitioners to operate at any point on the sensitivity/specificity trade-off curve without retraining. In high-stakes settings, a practitioner might choose to lower the decision threshold to maximize recall (catching every potential case at the expense of more false positives), whereas in resource-constrained settings they might raise it to reduce unnecessary follow-up referrals. The deterministic rule fixes this trade-off and cannot be adjusted. Second, the probabilistic output supports individual-level uncertainty communication: rather than simply being told "this individual is screen-positive," a clinician can be told "this individual has an estimated 94 percent probability of being screen-positive under the AQ-10 algorithm," which carries additional actionable information.

Why Logistic Regression Achieves Near-Perfect Performance: A Structural Analysis

The near-perfect AUC of Logistic Regression is the central finding of this project. It requires an explanation that goes beyond simply reporting the number, because misinterpreting it would lead to unjustified clinical claims.

The label is a function of the features. According to the dataset documentation, the AQ-10 screening label is generated automatically from the questionnaire: an individual receives a positive screening flag if and only if their total score exceeds 6. The total score is the unweighted sum of the ten binary item responses,

which are the same features used to train every model in this study. This relationship can be expressed formally as

$$Y = \mathbf{1}\left(\sum_{j=1}^{10} A_j > 6\right),$$

where A_j denotes the binary score for the j-th AQ-10 item and $\mathbf{1}(\cdot)$ is the indicator function, which returns 1 when the summed score exceeds the clinical threshold and 0 otherwise.

This means the outcome variable is, in the strict statistical sense, a deterministic function of the predictors. Any model capable of learning a monotone decision boundary based on the summed AQ-10 item responses can reconstruct this screening rule from the individual questionnaire items. Crucially, the `result` column (the raw AQ total) was dropped before any modeling, so the models do not have direct access to the sum; they must reconstruct it from the individual item scores.

Logistic Regression is well suited to this problem because the released label is based on a linear threshold of the AQ-10 item scores. A logistic regression on ten binary predictors learns a weighted sum of those predictors passed through a sigmoid function. When the true decision boundary is a linear threshold on the sum, which is precisely the case here, the logistic model is structurally matched to the problem. Complete (or near-complete) separation is therefore an expected consequence. Indeed, the AQ-only logistic regression fit exhibits the standard numerical signatures of complete separation: large unstable coefficient estimates, inflated standard errors, and p-values near 1.0. These are not evidence of a poorly specified model; they are the anticipated numerical behaviour when a linear model encounters a dataset it can classify without error. No coefficient table is needed to establish this interpretation; the structural argument is both sufficient and more reliable than numerically unstable maximum-likelihood estimates under separation.

Why elastic-net regularization does not change the result. An elastic-net sweep across alpha in {0, 0.5, 1} and lambda over a 30-point logarithmic grid produced virtually identical CV AUC of **1**, indistinguishable from the unregularized model. When model performance does not degrade under penalization, it indicates that the signal is strong, broadly distributed across features, and not concentrated in a small number of large unstable coefficients that regularization would shrink toward zero. This pattern is precisely what the AQ-10 scoring algorithm specifies: all ten items contribute approximately equally to the sum, so no single item dominates, and shrinking any subset of coefficients does not degrade prediction. The regularization sweep is therefore a useful diagnostic: it confirms that near-perfect LR performance is not a consequence of overfitting on a few noisy large coefficients but reflects a genuinely well-specified linear structure.

Why tree-based models add nothing. Random Forest and GBM achieve competitive but not superior AUC compared to LR. Decision trees and their ensembles are well suited to non-linear decision boundaries and complex feature interactions. When the true boundary is a linear sum threshold, ensemble methods add variance without bias reduction: they approximate a simple linear function using piecewise-constant representations, which is computationally expensive and no more accurate. The flat RF tuning curve observed across the full mtry range (2 through 8), and the near-identical GBM performance across all 16 hyperparameter combinations, are both consistent with this interpretation. The ensemble models are not failing; they are performing at ceiling. But the ceiling is imposed by the data structure, not by model capacity, and that ceiling is equally accessible to the much simpler logistic model.

Leakage versus legitimate structure. It is essential to distinguish two possible explanations for near-perfect performance. Label leakage occurs when the outcome variable is inadvertently included among the predictors, either directly (through a data column that encodes the label) or indirectly (through test-set information contaminating the training pipeline). Structural alignment occurs when the predictor space and the outcome share the same mathematical form, without any improper data sharing. This study eliminates leakage by design: the `result` column was dropped before any modeling, and all preprocessing and hyperparameter selection was conducted within the training fold only. What remains is structural alignment: the models are genuinely learning from behavioral item responses, but those responses are exactly what the label

was computed from. Table 5 quantifies this alignment by measuring how closely the deterministic cutoff rule (score > 6) reproduces the dataset label.

Table 5: Agreement between the AQ-10 cutoff rule (score > 6) and the dataset screening label, by group test set. Near-perfect agreement confirms the label is a near-deterministic function of the AQ total score, explaining why all models achieve near-ceiling performance.

Group	N_test	AQ_cutoff_accuracy	Perfect_agreement
Child	58	100.0%	TRUE
Adolescent	20	100.0%	TRUE
Adult	140	100.0%	TRUE
Combined	219	100.0%	TRUE

The agreement rates in Table 5 confirm the expected relationship: the released screening label is reproduced exactly by the AQ-10 rule (score > 6) across all four groups. The substantive conclusion is unambiguous: machine learning models trained on this dataset are learning to reconstruct the questionnaire-derived screening labels rather than predict an independent clinical outcome, because the released screening labels are derived directly from the AQ-10 item responses.

Implication for deployment. These findings do not invalidate the use of ML in ASD screening contexts, but they define its scope with precision. Within this dataset, the primary practical advantage of ML is the production of continuous probability estimates and flexible decision thresholds rather than improved agreement with the released screening labels. The models are not appropriate for: claiming independent clinical diagnostic capability, generalising to populations who have never been administered the AQ-10 questionnaire, or replacing specialist clinical assessment. Any deployment scenario that conflates near-perfect performance on this dataset with evidence of clinical validity would be methodologically unsound.

Conclusions

This study evaluated six statistical learning models (Logistic Regression, LDA, Naive Bayes, KNN, Random Forest, and GBM) for predicting the AQ-10 questionnaire-derived screening outcome across Child, Adolescent, and Adult datasets. While the original objective was to compare models across age groups, evaluate feature importance, and assess the contribution of demographic variables, the analysis ultimately revealed a more fundamental result. The consistently near-perfect performance of the best-performing models is explained by the construction of the released screening labels, which are derived directly from the AQ-10 questionnaire responses. Consequently, the primary contribution of this study extends beyond model comparison to demonstrating how the data-generation process fundamentally shapes the interpretation of predictive performance.

Model performance. All six models achieve strong AUC on the Combined dataset, with Logistic Regression and LDA reaching near-ceiling values. The performance ordering is consistent across age groups and evaluation metrics. Near-perfect performance of linear models is not surprising given the data structure: the screening label is a threshold applied to the unweighted sum of the ten item scores, making the decision boundary inherently linear. Non-linear ensembles (RF and GBM) offer no AUC advantage because there is no non-linear structure to exploit. The plateau in AUC across all model families is diagnostic evidence of a label structure that imposes a performance ceiling regardless of model complexity.

Feature importance. RF Mean Decrease in Accuracy and GBM Relative Influence converge on the same four most predictive items: A5 (social interaction), A9 (social ease), A4 (task switching), and A6 (pragmatic language). Demographic variables contribute no meaningful signal once behavioral items are included. The AQ-only feature set matches the full feature set on all models and all groups, confirming that demographic

characteristics are not necessary for reproducing the screening label. These rankings reflect association with the questionnaire-derived label, not causal pathways to ASD as a clinical condition.

ML vs clinical rule. The AQ-10 deterministic cutoff (score > 6) achieves high recall and competitive AUC across all groups, because the rule was designed to be sensitive and because AUC is driven by the continuous score's discriminative ability rather than any particular threshold. All trained ML models match this baseline closely. Within this dataset, the practical advantage of ML lies in producing calibrated probability estimates and supporting flexible threshold adjustment rather than in improving agreement with the released labels.

Label structure and the broader lesson. The central finding is that logistic regression achieves near-perfect performance because it is structurally matched to a problem where the outcome is a linear threshold of the same item scores used as predictors. Elastic-net regularization, the complete-separation behaviour of the LR fit, and the label-agreement analysis all independently confirm this interpretation. More broadly, this study illustrates that outstanding predictive performance should always be interpreted in light of the mechanism by which the target variable was generated. Model evaluation cannot be separated from the data-generation process. Near-perfect predictive performance is only meaningful when the target represents an independent outcome rather than a deterministic transformation of the predictors, and this distinction carries direct practical consequences for how results are reported, validated, and deployed.

Limitations. Because the outcome is derived directly from the AQ-10 responses, the reported performance primarily reflects agreement with the questionnaire scoring algorithm rather than diagnostic accuracy for ASD. The single 80/20 split introduces sampling variability in all point estimates, with the adolescent group (n = 104) most affected given its small size. Selection bias is structural: only individuals who completed the AQ-10 appear in the data, so the models cannot be expected to generalise to unscreened populations. GBM was tuned over a limited grid; broader exploration might identify marginally different hyperparameter configurations, though the structural analysis predicts no meaningful performance improvement.

Future directions. The most important extension is to evaluate these models on datasets where the outcome is an externally validated clinician-confirmed diagnosis rather than a questionnaire-derived label. This would separate algorithmic fidelity from clinical validity and provide a genuine test of whether the item-level behavioral signals have independent diagnostic value. Additional work should focus on external validation using independent screening cohorts, assessing whether the observed structural relationship generalizes across more diverse populations, and determining whether similar findings hold for related autism screening instruments such as the AQ-50 or ADOS-2.

Appendix

A: Phase 1 Full CV Results

Table 6: Complete 5-fold cross-validation results across all groups and models.

Model	AUC	Acc	Rec	Prec	Spec	F1
Child						
Logistic Reg.	0.998	0.966	0.982	0.949	0.950	0.965
LDA	0.986	0.915	0.956	0.878	0.876	0.915
Random Forest	0.986	0.953	0.938	0.964	0.967	0.951
Naive Bayes	0.969	0.876	0.832	0.904	0.917	0.866
KNN (k=5)	0.927	0.863	0.965	0.796	0.769	0.872
Adolescent						
Random Forest	0.979	0.893	0.980	0.862	0.758	0.917
LDA	0.977	0.893	0.980	0.862	0.758	0.917
Naive Bayes	0.933	0.798	0.765	0.886	0.848	0.821
KNN (k=5)	0.874	0.786	0.980	0.746	0.485	0.847

Table 6: Complete 5-fold cross-validation results across all groups and models. *(continued)*

Model	AUC	Acc	Rec	Prec	Spec	F1
Logistic Reg.	0.856	0.821	0.882	0.833	0.727	0.857
Adult						
Logistic Reg.	0.997	0.993	1.000	0.974	0.990	0.987
Random Forest	0.994	0.961	0.875	0.978	0.993	0.924
LDA	0.991	0.956	0.921	0.915	0.968	0.918
KNN (k=5)	0.956	0.910	0.849	0.822	0.932	0.835
Naive Bayes	0.951	0.929	0.901	0.846	0.939	0.873
Combined						
Logistic Reg.	0.998	0.998	1.000	0.994	0.996	0.997
LDA	0.998	0.975	0.987	0.945	0.968	0.966
Random Forest	0.995	0.960	0.917	0.970	0.984	0.943
Naive Bayes	0.960	0.902	0.851	0.873	0.931	0.862
KNN (k=5)	0.960	0.890	0.908	0.808	0.880	0.855

B: AQ-10 Item Reference

Table 7: AQ-10 behavioral item descriptions.

Item	Short	Behavioral Description
A1_Score	A1	Notices small sounds others don't
A2_Score	A2	Focuses on details rather than the whole picture
A3_Score	A3	Able to multitask easily (reverse-scored)
A4_Score	A4	Switches between tasks easily (reverse-scored)
A5_Score	A5	Doesn't know how to interact socially (reverse-scored)
A6_Score	A6	Difficulty reading between the lines in conversation
A7_Score	A7	Notices patterns in numbers or objects
A8_Score	A8	Finds it easy to do more than one thing at once (reverse-scored)
A9_Score	A9	Finds social situations easy (reverse-scored)
A10_Score	A10	Has difficulty working out people's intentions

C: Phase 1 Full Test Results by Group

Table 8: Phase 1 test set results across all age groups.

Model	AUC	Acc	Rec	Prec	F1
Child					
Logistic Reg.	1.000	1.000	1.000	1.000	1.000
LDA	1.000	0.983	1.000	0.966	0.982
Random Forest	0.983	0.948	0.964	0.931	0.947
KNN (tuned)	0.924	0.776	0.964	0.692	0.806
Adolescent					
Logistic Reg.	1.000	0.900	1.000	0.857	0.923
Random Forest	0.990	0.900	1.000	0.857	0.923
LDA	0.979	0.950	1.000	0.923	0.960
KNN (tuned)	0.906	0.750	0.917	0.733	0.815
Adult					
Logistic Reg.	1.000	1.000	1.000	1.000	1.000
LDA	0.997	0.971	0.973	0.923	0.947
Random Forest	0.996	0.964	0.892	0.971	0.930
KNN (tuned)	0.968	0.900	0.838	0.795	0.816
Combined					
Logistic Reg.	1.000	1.000	1.000	1.000	1.000

LDA	1.000	0.977	0.987	0.951	0.969
Random Forest	0.995	0.968	0.923	0.986	0.954
KNN (tuned)	0.950	0.886	0.846	0.835	0.841

D: Source Code – Phase 1 (`asd_analysis.R`)

```r
# asd_analysis.R
# ISYE 7406 - Midterm: ASD Risk Screening Across Age Groups
# Abdurrahman Sabbahi | asabbahi3@gatech.edu

library(tidyverse); library(caret); library(randomForest)
library(e1071); library(MASS); library(pROC)
library(corrplot); library(glmnet); library(gridExtra)

select <- dplyr::select; filter <- dplyr::filter; set.seed(42)

# --- Paths ---
script_dir <- tryCatch(dirname(normalizePath(sys.frame(1)$ofile)), error = function(e) getwd())
data_dir   <- file.path(dirname(script_dir), "ASD Different Age Group Dataset")
fig_dir    <- file.path(script_dir, "figures")
dir.create(fig_dir, recursive = TRUE, showWarnings = FALSE)
savefig <- function(name, w = 8, h = 5)
  ggsave(file.path(fig_dir, paste0(name, ".png")), width = w, height = h, dpi = 150)

# --- 1. Load & Clean ---
load_clean <- function(file, label) {
  df <- read.csv(file.path(data_dir, file),
                 stringsAsFactors = FALSE, na.strings = c("", "?", " "))
  names(df) <- trimws(names(df))
  names(df)[names(df) %in% c("Class.ASD", "Class/ASD")] <- "ASD"
  df %>%
    mutate(across(where(is.character), trimws),
           age_group = label,
           age       = suppressWarnings(as.numeric(age))) %>%
    select(-any_of(c("id", "age_desc", "result")))
}

child <- load_clean("Autism_Child_Data.csv",      "Child")
teen  <- load_clean("Autism_Adolescent_Data.csv", "Adolescent")
adult <- load_clean("Autism_Adult_Data.csv",       "Adult")
comb  <- bind_rows(child, teen, adult)
aq_cols <- paste0("A", 1:10, "_Score")

# --- 2. Train/Test Split (80/20 stratified) ---
group_list <- list(Child = child, Adolescent = teen, Adult = adult, Combined = comb)

impute_age <- function(train_df, test_df) {
  train_med <- median(train_df$age, na.rm = TRUE)
  train_df$age[is.na(train_df$age)] <- train_med
  test_df$age[is.na(test_df$age)]   <- train_med
  list(train = train_df, test = test_df)
}
```

```r
splits <- lapply(group_list, function(df) {
  idx <- createDataPartition(df$ASD, p = 0.8, list = FALSE)
  impute_age(df[idx, ], df[-idx, ])
})

# --- 3. Feature Encoding ---
NOM_COLS <- c("ethnicity", "contry_of_res", "relation")

encode_binary <- function(df) {
  df %>% mutate(
    ASD_bin         = as.integer(ASD == "YES"),
    gender          = as.integer(tolower(gender)          == "m"),
    jundice         = as.integer(tolower(jundice)         == "yes"),
    austim          = as.integer(tolower(austim)          == "yes"),
    used_app_before = as.integer(tolower(used_app_before) == "yes"),
    ethnicity       = replace_na(ethnicity,     "Unknown"),
    contry_of_res   = replace_na(contry_of_res, "Unknown"),
    relation        = replace_na(relation,      "Unknown")
  )
}

build_dummy_schema <- function(train_df) {
  enc <- encode_binary(train_df)
  lapply(NOM_COLS, function(col) { lvls <- sort(unique(enc[[col]])); lvls[-1] }) %>%
    setNames(NOM_COLS)
}

apply_dummies <- function(df, schema) {
  df_enc <- encode_binary(df)
  dummy_frames <- lapply(NOM_COLS, function(col) {
    wanted <- schema[[col]]; vals <- df_enc[[col]]
    mm <- as.data.frame(sapply(wanted, function(lv) as.integer(vals == lv)))
    setNames(mm, gsub("[^A-Za-z0-9_]", "_", paste0(col, "_", wanted)))
  })
  df_out <- df_enc %>% select(-all_of(NOM_COLS))
  do.call(cbind, c(list(df_out), dummy_frames))
}

feat_aq <- aq_cols
make_Xy <- function(df, cols, schema) {
  enc  <- apply_dummies(df, schema)
  keep <- intersect(cols, names(enc))
  X    <- enc %>% select(all_of(keep)) %>%
    mutate(across(everything(), ~replace_na(as.numeric(.x), 0)))
  y    <- factor(enc$ASD_bin, levels = c(0, 1), labels = c("NO", "YES"))
  list(X = as.data.frame(X), y = y)
}

ds <- lapply(splits, function(sp) {
  schema   <- build_dummy_schema(sp$train)
  demo_bin <- c("age", "gender", "jundice", "austim", "used_app_before")
  dummy_cols <- grep("^(ethnicity|relation)_",
                     names(apply_dummies(sp$train, schema)), value = TRUE)
```

```
  f_full <- c(aq_cols, intersect(c(demo_bin, dummy_cols),
                                 names(apply_dummies(sp$train, schema))))
  list(
    schema = schema,
    full   = list(train = make_Xy(sp$train, f_full,   schema),
                  test  = make_Xy(sp$test,  f_full,   schema)),
    aq     = list(train = make_Xy(sp$train, feat_aq,  schema),
                  test  = make_Xy(sp$test,  feat_aq,  schema))
  )
})

# --- 4. Descriptive Statistics ---
desc <- map_dfr(group_list, function(df) {
  aq_sum <- rowSums(df[, aq_cols], na.rm = TRUE)
  tibble(N           = nrow(df),
         `ASD+ (%)` = round(mean(df$ASD == "YES") * 100, 1),
         `AQ Mean`  = round(mean(aq_sum), 2),
         `AQ SD`    = round(sd(aq_sum),   2))
}, .id = "Dataset")

# --- 5. EDA ---
aq_long <- map_dfr(list(Child = child, Adolescent = teen, Adult = adult),
                   ~.x %>% mutate(AQ = rowSums(across(all_of(aq_cols)))),
                   .id = "Dataset") %>%
  mutate(Dataset = factor(Dataset, levels = c("Child", "Adolescent", "Adult")))

comb_enc <- apply_dummies(splits$Combined$train, ds$Combined$schema)
corr_mat <- comb_enc %>%
  select(all_of(c(aq_cols, "age", "gender", "jundice", "austim", "ASD_bin"))) %>%
  rename_with(~str_replace(.x, "_Score", "")) %>% rename(ASD = ASD_bin) %>%
  cor(use = "complete.obs")

# --- 6. PCA ---
Xtr_pca <- ds$Combined$full$train$X
pca_res  <- prcomp(scale(Xtr_pca), center = FALSE, scale. = FALSE)
var_exp  <- pca_res$sdev^2 / sum(pca_res$sdev^2)
cum_var  <- cumsum(var_exp)

# --- 7. Chi-Square Feature Ranking ---
chi2_res <- map_dfr(c(aq_cols, "gender", "jundice", "austim"), function(col) {
  ct <- table(comb[[col]], comb$ASD)
  if (any(dim(ct) < 2)) return(NULL)
  tst <- chisq.test(ct, simulate.p.value = FALSE)
  tibble(Feature = col, Chi2 = round(tst$statistic, 2), p = round(tst$p.value, 5))
}) %>% arrange(desc(Chi2))

# --- 8. 5-Fold Cross-Validation ---
cv_ctrl <- trainControl(method = "cv", number = 5,
                        classProbs = TRUE, summaryFunction = twoClassSummary,
                        savePredictions = "final")

model_specs <- list(
  "Logistic Reg." = list(method = "glm",  family = binomial(),
```

```
                          preProc = c("center", "scale")),
  "LDA"           = list(method = "lda",
                          preProc = c("nzv", "center", "scale", "pca")),
  "Naive Bayes"   = list(method = "nb",
                          preProc = c("nzv", "center", "scale"),
                          tuneGrid = data.frame(fL = 1, usekernel = FALSE, adjust = 1)),
  "KNN (k=5)"     = list(method = "knn",  tuneGrid = data.frame(k = 5),
                          preProc = c("center", "scale")),
  "Random Forest" = list(method = "rf",   tuneGrid = data.frame(mtry = 3),
                          preProc = c("center", "scale"))
)

run_cv <- function(ds_train, label) {
  map_dfr(names(model_specs), function(mn) {
    sp   <- model_specs[[mn]]
    args <- list(x = ds_train$X, y = ds_train$y,
                 method = sp$method, trControl = cv_ctrl, metric = "ROC",
                 preProcess = sp$preProc)
    if (!is.null(sp$family))   args$family   <- sp$family
    if (!is.null(sp$tuneGrid)) args$tuneGrid <- sp$tuneGrid
    fit <- tryCatch(do.call(train, args),
                    error = function(e) { message("skip ", mn, ": ", e$message); NULL })
    if (is.null(fit) || is.null(fit$pred)) return(NULL)
    best    <- fit$results[which.max(fit$results$ROC), ]
    p_clean <- fit$pred[!is.na(fit$pred$pred) & !is.na(fit$pred$obs), ]
    tn <- sum(p_clean$pred == "NO" & p_clean$obs == "NO")
    fp <- sum(p_clean$pred == "YES" & p_clean$obs == "NO")
    tibble(Group = label, Model = mn,
           AUC  = round(best$ROC, 3),
           Acc  = round(mean(p_clean$pred == p_clean$obs), 3),
           Rec  = round(recall(p_clean$pred,    p_clean$obs, relevant = "YES"), 3),
           Prec = round(precision(p_clean$pred, p_clean$obs, relevant = "YES"), 3),
           F1   = round(F_meas(p_clean$pred,    p_clean$obs, relevant = "YES"), 3),
           Spec = if ((tn + fp) > 0) round(tn / (tn + fp), 3) else NA_real_)
  })
}

cv_res <- map_dfr(names(ds), function(nm) run_cv(ds[[nm]]$full$train, nm))
cv_res$Group <- factor(cv_res$Group, levels = c("Child","Adolescent","Adult","Combined"))

# --- 10. Hyperparameter Tuning (per group) ---
tune_group <- function(nm) {
  Xg <- ds[[nm]]$full$train$X; yg <- ds[[nm]]$full$train$y
  knn_fit <- train(x = Xg, y = yg, method = "knn", trControl = cv_ctrl, metric = "ROC",
                   preProcess = c("center","scale"), tuneGrid = data.frame(k = 1:15))
  rf_fit  <- train(x = Xg, y = yg, method = "rf",  trControl = cv_ctrl, metric = "ROC",
                   preProcess = c("center","scale"), tuneGrid = data.frame(mtry = 2:8))
  list(knn = knn_fit, rf = rf_fit,
       best_k    = knn_fit$bestTune$k,   best_mtry = rf_fit$bestTune$mtry,
       knn_auc   = round(max(knn_fit$results$ROC), 3),
       rf_auc    = round(max(rf_fit$results$ROC),  3))
}
```

```
tune_res <- setNames(lapply(names(ds), function(nm) tune_group(nm)), names(ds))
Xc <- ds$Combined$full$train$X;  yc <- ds$Combined$full$train$y
knn_best_k   <- tune_res$Combined$best_k
rf_best_mtry <- tune_res$Combined$best_mtry

# --- 11. Regularized Logistic Regression ---
glmnet_grid <- expand.grid(alpha  = c(0, .5, 1),
                           lambda = 10^seq(-4, 0, length.out = 30))
glmnet_fit  <- train(x = Xc, y = yc, method = "glmnet", trControl = cv_ctrl,
                     metric = "ROC", preProcess = c("center","scale"),
                     tuneGrid = glmnet_grid)
glmnet_best <- glmnet_fit$bestTune
glmnet_auc  <- round(max(glmnet_fit$results$ROC), 3)

# --- 12. RF Variable Importance ---
pp_sc  <- preProcess(Xc, method = c("center","scale"))
rf_imp <- randomForest(x = predict(pp_sc, Xc), y = yc, ntree = 500, importance = TRUE)
imp_df <- importance(rf_imp, type = 1) %>%
  as.data.frame() %>% rownames_to_column("Feature") %>%
  rename(MDA = MeanDecreaseAccuracy) %>% arrange(desc(MDA))

# --- 13. Full vs AQ-Only Feature Set Comparison ---
compare_res <- map_dfr(names(ds), function(nm) {
  bind_rows(
    run_cv(ds[[nm]]$full$train, nm) %>% mutate(FeatSet = "Full"),
    run_cv(ds[[nm]]$aq$train,   nm) %>% mutate(FeatSet = "AQ-Only")
  )
})
comp_comb <- compare_res %>% filter(Group == "Combined") %>%
  mutate(FeatSet = factor(FeatSet, levels = c("Full","AQ-Only")))

# --- 14. Final Test Set Evaluation ---
eval_on_test <- function(nm) {
  tr <- ds[[nm]]$full$train; te <- ds[[nm]]$full$test
  specs <- list(
    "Logistic Reg." = list(method = "glm",  family = binomial(),
                           preProc = c("center","scale"), tuneGrid = NULL),
    "LDA"           = list(method = "lda",
                           preProc = c("nzv","center","scale","pca"), tuneGrid = NULL),
    "KNN (tuned)"   = list(method = "knn",  preProc = c("center","scale"),
                           tuneGrid = data.frame(k = tune_res[[nm]]$best_k)),
    "Random Forest" = list(method = "rf",   preProc = c("center","scale"),
                           tuneGrid = data.frame(mtry = tune_res[[nm]]$best_mtry))
  )
  map_dfr(names(specs), function(mn) {
    sp     <- specs[[mn]]
    pp_fit <- preProcess(tr$X, method = sp$preProc)
    Xtr_pp <- predict(pp_fit, tr$X); Xte_pp <- predict(pp_fit, te$X)
    args <- list(x = Xtr_pp, y = tr$y, method = sp$method,
                 trControl = trainControl(method = "none", classProbs = TRUE))
    if (!is.null(sp$family))   args$family   <- sp$family
    if (!is.null(sp$tuneGrid)) args$tuneGrid <- sp$tuneGrid
    fit <- tryCatch(do.call(train, args), error = function(e) NULL)
```

```
    if (is.null(fit)) return(NULL)
    pc  <- predict(fit, Xte_pp)
    pp2 <- predict(fit, Xte_pp, type = "prob")[, "YES"]
    tibble(Group = nm, Model = mn,
           AUC  = round(auc(roc(as.integer(te$y == "YES"), pp2, quiet = TRUE)), 3),
           Acc  = round(mean(pc == te$y), 3),
           Rec  = round(recall(pc,    te$y, relevant = "YES"), 3),
           Prec = round(precision(pc, te$y, relevant = "YES"), 3),
           F1   = round(F_meas(pc,    te$y, relevant = "YES"), 3))
  })
}

test_res <- map_dfr(names(ds), eval_on_test)
test_res$Group <- factor(test_res$Group, levels = c("Child","Adolescent","Adult","Combined"))
```

E: Source Code – Phase 2 (`asd_analysis_phase2.R`)

```
# asd_analysis_phase2.R
# ISYE 7406 - Final Report Phase 2
# Abdurrahman Sabbahi | asabbahi3@gatech.edu

library(tidyverse); library(caret); library(randomForest)
library(e1071); library(MASS); library(pROC)
library(corrplot); library(glmnet); library(gridExtra)
library(gbm); library(boot)

select <- dplyr::select; filter <- dplyr::filter; set.seed(42)
source("asd_analysis.R", local = TRUE)

# --- AQ-10 Behavioral Item Descriptions ---
aq_descriptions <- tribble(
  ~Item,       ~Short, ~Description,
  "A1_Score",  "A1",   "Notices small sounds others don't",
  "A2_Score",  "A2",   "Focuses on details rather than the whole picture",
  "A3_Score",  "A3",   "Able to multitask easily (reverse-scored)",
  "A4_Score",  "A4",   "Switches between tasks easily (reverse-scored)",
  "A5_Score",  "A5",   "Doesn't know how to interact socially (reverse-scored)",
  "A6_Score",  "A6",   "Difficulty reading between the lines in conversation",
  "A7_Score",  "A7",   "Notices patterns in numbers or objects",
  "A8_Score",  "A8",   "Finds it easy to do more than one thing at once (reverse-scored)",
  "A9_Score",  "A9",   "Finds social situations easy (reverse-scored)",
  "A10_Score", "A10",  "Has difficulty working out people's intentions"
)

# --- AQ-10 Clinical Cutoff Rule (score > 6) ---
aq_cutoff_eval <- map_dfr(names(splits), function(nm) {
  te_raw <- splits[[nm]]$test
  aq_sum <- rowSums(te_raw[, aq_cols], na.rm = TRUE)
  y_true <- as.integer(te_raw$ASD == "YES")
  y_pred <- as.integer(aq_sum > 6)
  tp <- sum(y_pred == 1 & y_true == 1); tn <- sum(y_pred == 0 & y_true == 0)
```

```
  fp <- sum(y_pred == 1 & y_true == 0); fn <- sum(y_pred == 0 & y_true == 1)
  roc_obj <- roc(y_true, aq_sum, quiet = TRUE)
  tibble(Group = nm, Model = "AQ-10 Cutoff (>6)",
         AUC   = round(as.numeric(auc(roc_obj)), 3),
         Acc   = round((tp + tn) / length(y_true), 3),
         Rec   = round(tp / (tp + fn), 3),
         Prec  = round(tp / (tp + fp), 3),
         Spec  = round(tn / (tn + fp), 3),
         F1    = round(2 * tp / (2 * tp + fp + fn), 3))
})

# --- GBM: CV + Test + Relative Influence ---
gbm_grid <- expand.grid(n.trees = c(100, 200), interaction.depth = c(2, 4),
                        shrinkage = c(0.05, 0.1), n.minobsinnode = 10)
cv_ctrl_gbm <- trainControl(method = "cv", number = 5,
                            classProbs = TRUE, summaryFunction = twoClassSummary,
                            savePredictions = "final")

run_gbm_cv <- function(ds_train, label) {
  fit <- tryCatch(
    train(x = ds_train$X, y = ds_train$y, method = "gbm",
          trControl = cv_ctrl_gbm, metric = "ROC",
          preProcess = c("center","scale"), tuneGrid = gbm_grid, verbose = FALSE),
    error = function(e) NULL)
  if (is.null(fit) || is.null(fit$pred)) return(NULL)
  best    <- fit$results[which.max(fit$results$ROC), ]
  p_clean <- fit$pred %>%
    filter(n.trees == fit$bestTune$n.trees,
           interaction.depth == fit$bestTune$interaction.depth,
           shrinkage == fit$bestTune$shrinkage) %>%
    filter(!is.na(pred) & !is.na(obs))
  tn <- sum(p_clean$pred == "NO" & p_clean$obs == "NO")
  fp <- sum(p_clean$pred == "YES" & p_clean$obs == "NO")
  tibble(Group = label, Model = "GBM",
         AUC  = round(best$ROC, 3),
         Acc  = round(mean(p_clean$pred == p_clean$obs), 3),
         Rec  = round(recall(p_clean$pred,    p_clean$obs, relevant = "YES"), 3),
         Prec = round(precision(p_clean$pred, p_clean$obs, relevant = "YES"), 3),
         F1   = round(F_meas(p_clean$pred,    p_clean$obs, relevant = "YES"), 3),
         Spec = if ((tn + fp) > 0) round(tn / (tn + fp), 3) else NA_real_)
}

gbm_cv_res <- map_dfr(names(ds), function(nm) run_gbm_cv(ds[[nm]]$full$train, nm))

eval_gbm_test <- function(nm) {
  tr <- ds[[nm]]$full$train; te <- ds[[nm]]$full$test
  cv_fit <- tryCatch(
    train(x = tr$X, y = tr$y, method = "gbm", trControl = cv_ctrl_gbm, metric = "ROC",
          preProcess = c("center","scale"), tuneGrid = gbm_grid, verbose = FALSE),
    error = function(e) NULL)
  best_tune <- if (!is.null(cv_fit)) cv_fit$bestTune else gbm_grid[1, ]
  pp_fit <- preProcess(tr$X, method = c("center","scale"))
  Xtr_pp <- predict(pp_fit, tr$X); Xte_pp <- predict(pp_fit, te$X)
```

```
  fit <- tryCatch(
    train(x = Xtr_pp, y = tr$y, method = "gbm",
          trControl = trainControl(method = "none", classProbs = TRUE),
          tuneGrid = best_tune, verbose = FALSE),
    error = function(e) NULL)
  if (is.null(fit)) return(NULL)
  pc  <- predict(fit, Xte_pp)
  pp2 <- predict(fit, Xte_pp, type = "prob")[, "YES"]
  tibble(Group = nm, Model = "GBM",
         AUC  = round(auc(roc(as.integer(te$y == "YES"), pp2, quiet = TRUE)), 3),
         Acc  = round(mean(pc == te$y), 3),
         Rec  = round(recall(pc,    te$y, relevant = "YES"), 3),
         Prec = round(precision(pc, te$y, relevant = "YES"), 3),
         F1   = round(F_meas(pc,    te$y, relevant = "YES"), 3))
}

gbm_test_res <- map_dfr(names(ds), eval_gbm_test)

pp_gbm   <- preProcess(Xc, method = c("center","scale"))
gbm_final <- train(x = predict(pp_gbm, Xc), y = yc, method = "gbm",
                   trControl = trainControl(method = "none", classProbs = TRUE),
                   tuneGrid  = data.frame(n.trees = 200, interaction.depth = 4,
                                          shrinkage = 0.05, n.minobsinnode = 10),
                   verbose = FALSE)
gbm_rel_inf <- summary(gbm_final$finalModel, plotit = FALSE) %>%
  as_tibble() %>% rename(Feature = var, RelInf = rel.inf) %>%
  arrange(desc(RelInf)) %>%
  left_join(aq_descriptions %>% select(Item, Description), by = c("Feature" = "Item")) %>%
  mutate(RelInf = round(RelInf, 2))

# --- Combined Comparison Table ---
full_test_res <- bind_rows(
  test_res %>% mutate(Model = as.character(Model)),
  gbm_test_res %>% mutate(Model = as.character(Model))
) %>% mutate(Group = factor(Group, levels = c("Child","Adolescent","Adult","Combined")))

comparison_vs_cutoff <- bind_rows(
  full_test_res,
  aq_cutoff_eval %>% select(Group, Model, AUC, Acc, Rec, Prec, F1)
) %>% arrange(Group, desc(AUC))

# --- Label Leakage Investigation ---
leakage_check <- map_dfr(names(splits), function(nm) {
  te_raw <- splits[[nm]]$test
  aq_sum <- rowSums(te_raw[, aq_cols], na.rm = TRUE)
  y_true <- as.integer(te_raw$ASD == "YES")
  tibble(Group              = nm,
         N_test             = nrow(te_raw),
         AQ_cutoff_accuracy = round(mean(as.integer(aq_sum > 6) == y_true), 4),
         Perfect_agreement  = mean(as.integer(aq_sum > 6) == y_true) == 1.0)
})

# --- Enhanced RF Importance with Item Descriptions ---
```

```r
imp_df_annotated <- imp_df %>%
  left_join(aq_descriptions %>% select(Item, Short, Description),
            by = c("Feature" = "Item")) %>%
  mutate(Label = ifelse(!is.na(Short), paste0(Short, ": ", Description), Feature))
```

References

Thabtah, F. (2017). *ASD Screening Datasets*. UCI Machine Learning Repository.

Baron-Cohen, S. et al. (2001). The Autism-Spectrum Quotient (AQ). *Journal of Autism and Developmental Disorders, 31*(1), 5–17.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.

Breiman, L. (2001). Random Forests. *Machine Learning, 45*(1), 5–32.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232.

Kuhn, M. (2008). Building Predictive Models in R Using the caret Package. *Journal of Statistical Software, 28*(5).
